Human Verification: `Containers.Aggregation`

Aggregation as a Routing Cofunctor

Machine-generated from `lean/Aggregation.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Aggregation` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file verifies Construction `con:aggregation` (§9 of the paper): a **routing policy** is a cofunctor (directed-container morphism) from the concrete 11-facility Sawasdee Cellars network to a 2-node aggregate network (Australia → Thailand).

Contents

1 Overview **1**

2 The aggregate network **2**
2.1 Inductive type: `Region` . . . 2
2.2 Inductive type: `AgLeg` . . . 2
2.3 Definition: `GAgg` . . . 2

3 The routing policy **2**
3.1 Definition: `region` (shape map) . . . 2
3.2 Definition: `lift` (position lift) . . . 3
3.3 Theorem: `routingM0` (Law M0 — shape compatibility) . . . 3
3.4 Theorem: `routingM1` (Law M1 — root preservation) . . . 4
3.5 Theorem: `routingM2` (Law M2 — shift compatibility) . . . 4
3.6 Definition: `routingPolicy` . . . 5

4 Translation conventions **5**

1 Overview

A *directed-container morphism* (cofunctor) from $\mathcal{D}$ to $\mathcal{E}$ consists of:

- a map on shapes $u : \mathcal{D}.\text{Shape} \to \mathcal{E}.\text{Shape}$, and
- a *contravariant* position lift: $\ell : (s : \mathcal{D}.\text{Shape}) \to \mathcal{E}.\text{Pos}(u(s)) \to \mathcal{D}.\text{Pos}(s)$,

satisfying three laws M0, M1, M2 (shape compatibility, root preservation, shift compatibility).

Here $\mathcal{D}$ is the Sawasdee Cellars network and $\mathcal{E}$ is the 2-node aggregate network. The shape map sends each facility to its region (Australia or Thailand); the lift sends each aggregate route to a canonical concrete route for that facility.

2 The aggregate network

2.1 Inductive type: `Region`

Definition 2.1 (Aggregate regions)**.** The 2-node aggregate graph has two nodes: `australia` and `thailand`, representing the two geographical regions of the wine supply chain.

Lean.
```
inductive Region | australia | thailand
  deriving DecidableEq, Repr
```

Haskell.
```
data Region = Australia | Thailand deriving (Eq, Show)
```

2.2 Inductive type: `AgLeg`

Definition 2.2 (Aggregate leg)**.** There is a single aggregate leg `au2th` representing the sea-freight connection from Australia to Thailand.

Lean.
```
inductive AgLeg | au2th
  deriving DecidableEq, Repr
```

Haskell.
```
data AgLeg = Au2th deriving (Eq, Show)
```

2.3 Definition: `GAgg`

Definition 2.3 (2-node aggregate digraph)**.** G_{agg} is the digraph with $V = \{\text{australia}, \text{thailand}\}$, $E = \{\text{au2th}\}$, $\text{src}(\text{au2th}) = \text{australia}$, $\text{tgt}(\text{au2th}) = \text{thailand}$.

Its network container $\mathcal{N}(G_{\text{agg}})$ has:

- $\text{Pos}(\text{australia}) = \{(\text{australia}, \varepsilon),\ (\text{thailand}, \text{au2th})\}$ — two aggregate routes;
- $\text{Pos}(\text{thailand}) = \{(\text{thailand}, \varepsilon)\}$ — one aggregate route (sink).

Lean.
```
@[reducible] def GAgg : Digraph where
  V := Region
  E := AgLeg
  src := agSrc
  tgt := agTgt
```

Haskell.
```
gAgg :: Digraph Region AgLeg
gAgg = Digraph { src = \Au2th → Australia
               , tgt = \Au2th → Thailand }
```

3 The routing policy

3.1 Definition: `region` (shape map)

Definition 3.1 (Region assignment)**.** Each facility belongs to a region:

$$\begin{aligned} \text{region}(v) &= \text{australia} \quad \text{for } v \in \{\text{vineyard}_i,\ \text{sawasdee},\ \text{melbournePort}\},\\ \text{region}(v) &= \text{thailand} \quad \text{for } v \in \{\text{thaiPort},\ \text{distributor}_i,\ \text{retail}_i\}. \end{aligned}$$

This is the shape map u of the routing-policy cofunctor.

Lean.

```
@[reducible] def region : Facility → Region
  | .vineyard1 | .vineyard2 | .vineyard3 ⇒ australia
  | .sawasdee | .melbournePort           ⇒ australia
  | .thaiPort                            ⇒ thailand
  | .distributor1 | .distributor2        ⇒ thailand
  | .retail1 | .retail2 | .retail3       ⇒ thailand
```

Haskell.

```
region :: Facility → Region
region Vineyard1    = Australia
region Vineyard2    = Australia
region Vineyard3    = Australia
region Sawasdee     = Australia
region MelbournePort = Australia
region _            = Thailand
```

3.2 Definition: `lift` (position lift)

Definition 3.2 (Canonical concrete route lift)**.** The **lift** $\ell(f, \rho)$ maps an aggregate route $\rho : \mathcal{N}(G_{\text{agg}}).\text{Pos}(\text{region}(f))$ to a canonical concrete route out of facility f.

- If $\rho = (\text{australia}, \varepsilon)$ (stay in Australia): $\ell(f, \rho) = (f, \varepsilon)$ for any Australian facility f.
- If $\rho = (\text{thailand}, \text{au2th})$ (cross to Thailand): $\ell(f, \rho)$ is the canonical route from f to Thai Port via the fixed outbound path for f.
- For Thai facilities, $\text{Pos}(\text{region}(f)) = \{(\text{thailand}, \varepsilon)\}$, so $\ell(f, \rho) = (f, \varepsilon)$.

Lean.

```
def lift : (f : Facility) → AgPos (region f) → Route f
  | .vineyard1, ar ⇒ match ar with
      | ⟨australia, .nil _⟩ ⇒ ⟨_, .nil _⟩
      | ⟨thailand,  _⟩      ⇒
          alongLeg Leg.v1s (alongLeg Leg.smp (alongLeg Leg.sea ⟨_, .
              nil _⟩))
  -- ... (similarly for all 11 facilities)
```

Haskell.

```
lift :: Facility → AgRoute Region AgLeg → Route Facility Leg
lift Vineyard1 (Australia, []) = (Vineyard1, [])
lift Vineyard1 (Thailand, _)   = (ThaiPort, [V1s, Smp, Sea])
lift Vineyard2 (Australia, []) = (Vineyard2, [])
lift Vineyard2 (Thailand, _)   = (ThaiPort, [V2s, Smp, Sea])
-- ... similarly for all facilities
lift f         (Thailand, [])  = (f, [])  -- Thai facilities: stay put
```

Fidelity note. The Lean type of `lift` is genuinely dependent: the aggregate position type is $\mathcal{N}(G_{\text{agg}}).\text{Pos}(\text{region}(f))$, which varies with f. Lean can compute $\text{region}(f)$ concretely by case-splitting on f, which is why the definition pattern-matches on f before matching on the aggregate route.

3.3 Theorem: `routingM0` (Law M0 — shape compatibility)

Theorem 3.3 (M0: shape compatibility)**.** For all facilities f and aggregate routes $\rho : \mathcal{N}(G_{\text{agg}}).\text{Pos}(\text{region}(f))$:

$$\text{region}\big(\mathcal{N}(\text{graph}).\text{sub}(f,\ \ell(f, \rho))\big)\ =\ \mathcal{N}(G_{\text{agg}}).\text{sub}\big(\text{region}(f),\ \rho\big).$$

The endpoint of the concrete route (in the wine network) maps to the aggregate endpoint under `region`. Proved by cases on f and ρ.

Lean.
```
theorem routingM0 (f : Facility) (ar : AgPos (region f)) :
    region (Wine.network.sub f (lift f ar)) =
    (Digraph.network GAgg).sub (region f) ar
```

Haskell.
```
-- M0: region (sub_wine f (lift f ar)) = sub_agg (region f) ar
-- For all f, ar: the endpoint of the concrete route has the right
   region.
propM0 :: Facility → AgRoute Region AgLeg → Bool
propM0 f ar = region (endpoint (lift f ar)) == agEndpoint (region f)
   ar
```

3.4 Theorem: `routingM1` (Law M1 — root preservation)

Theorem 3.4 (M1: root preservation)**.** For all facilities f:

$$\ell(f,\ \mathcal{N}(G_{\text{agg}}).\text{root}(\text{region}(f)))\ =\ \mathcal{N}(\text{graph}).\text{root}(f).$$

Lifting the aggregate empty route gives the concrete empty route.

Lean.
```
theorem routingM1 (f : Facility) :
    lift f ((Digraph.network GAgg).root (region f)) =
    Wine.network.root f
```

Haskell.
```
-- M1: lift f (agRoot (region f)) = wineRoot f
-- Lifting the aggregate empty route is the concrete empty route.
propM1 :: Facility → Bool
propM1 f = lift f (agRoot (region f)) == wineRoot f
```

3.5 Theorem: `routingM2` (Law M2 — shift compatibility)

Theorem 3.5 (M2: shift compatibility)**.** For all f : Facility, aggregate routes $\rho : \mathcal{N}(G_{\text{agg}}).\text{Pos}(\text{region}(f))$, and $k : \mathcal{N}(G_{\text{agg}}).\text{Pos}(\mathcal{N}(G_{\text{agg}}).\text{sub}(\text{region}(f), \rho))$:

$$\ell(f,\ \mathcal{N}(G_{\text{agg}}).\text{shift}(\text{region}(f), \rho, k))\ =\ \mathcal{N}(\text{graph}).\text{shift}\Big(f,\ \ell(f,\rho),\ \ell\Big(\mathcal{N}(\text{graph}).\text{sub}(f, \ell(f,\rho)),\ \text{M0}^{-1}(k)\Big)\Big).$$

Lifting a concatenated aggregate route equals the concrete concatenation of the lifted segments.

Lean.
```
theorem routingM2 (f : Facility) (ar : AgPos (region f))
    (k : (Digraph.network GAgg).Pos
           ((Digraph.network GAgg).sub (region f) ar)) :
    lift f ((Digraph.network GAgg).shift (region f) ar k) =
    Wine.network.shift f (lift f ar)
      (lift (Wine.network.sub f (lift f ar))
            ((routingM0 f ar).symm cast k))
```

Fidelity note. The transport `(routingM0 f ar).symm cast k` (Lean notation: `.symm ▶ k`) coerces k along the M0 equality. In the concrete proof, M0 is `rfl` in all cases, so the transport reduces to the identity. The Haskell rendering omits this coercion entirely.

Haskell.

```haskell
-- M2: lift f (agShift (region f) ar k)
--       = wineShift f (lift f ar) (lift (wineSub f (lift f ar)) k)
-- (Omitting the transport; it is trivial in all concrete cases.)
propM2 :: Facility → AgRoute Region AgLeg
       → AgRoute Region AgLeg → Bool
propM2 f ar k =
  lift f (agShift (region f) ar k) ==
  wineShift f (lift f ar) (lift (wineSub f (lift f ar)) k)
```

3.6 Definition: `routingPolicy`

Definition 3.6 (Routing policy as a directed-container morphism)**.** The **routing policy** is the directed-container morphism (cofunctor)

$$\text{routingPolicy} : \text{DContMorphism}(\mathcal{N}(\text{graph}),\ \mathcal{N}(G_{\text{agg}}))$$

with shape map `region` and position lift `lift`, satisfying laws M0, M1, M2. This verifies Construction `con:aggregation`.

Lean.

```
def routingPolicy : DContMorphism Wine.network (Digraph.network GAgg)
    where
  onShapes := region
  onPos    := lift
  m0       := routingM0
  m1       := routingM1
  m2       := routingM2
```

Haskell.

```haskell
data DContMorphism d e = DContMorphism
  { onShapes :: Shape d → Shape e
  , onPos    :: (s :: Shape d) → Pos e (onShapes s) → Pos d s
  -- plus proofs of M0, M1, M2
  }

routingPolicy :: DContMorphism WineNetwork AggNetwork
routingPolicy = DContMorphism
  { onShapes = region
  , onPos    = lift
  }
```

Fidelity note. A directed-container morphism is a *cofunctor*: the position map goes *backwards*, from aggregate positions to concrete positions, just as a container morphism's position map is contravariant. The shape map goes forward (facility → region) while the position lift goes backward (aggregate route → concrete route).

4 Translation conventions

1. **Dependent position types.** The type of `lift` is $\Pi(f : \text{Facility}), \mathcal{N}(G_{\text{agg}}).\text{Pos}(\text{region}(f)) \to \text{Route}(f)$. The first argument determines the type of the aggregate position; this is a genuinely dependent type that Haskell cannot express directly.
2. **Transport in M2.** The cast `(routingM0 f ar).symm cast k` (Lean: `.symm ▶ k`) appears because the endpoint of `lift f ar` has type `Facility`, while k's domain is parametrised by a `Region`. The M0 equality converts between them. In all concrete cases, M0 is `rfl` and the transport is the identity.

3. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.